
Pantry Dashboard v2

From stretched mobile to a warm, intelligent pantry system

Cyrus Dioun · March 2026

The Problem — v1 Limitations

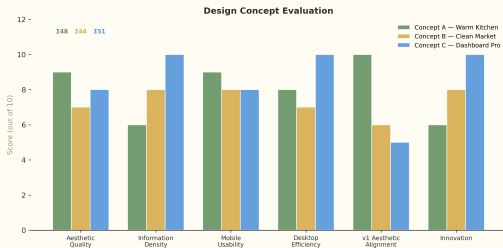
- **No category system** — 38/58 products have OFF data, unused
- **Desktop is stretched mobile** — no sidebar, no desktop layout
- **Bare empty/error states** — no retry, no skeleton loading
- **Location cycling hidden** — blind tap-to-rotate, no drop-down
- **Sort direction missing** — A→Z vs Z→A unclear
- **Kept:** warm palette, emoji system, expiry urgency



Failed to load pantry data

v1 desktop: stretched card grid, no sidebar, no category filters

Design Process — Three Concepts Evaluated



Decision: A + C Hybrid (“Warm Dashboard”)

A: cream/sage palette, cards, pill scroll **C:** sidebar, stat cards, sort direction

Not C's dark sidebar — too clinical for a kitchen app



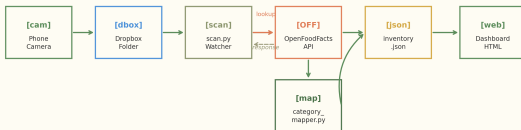
Concept A — Warm Kitchen



Concept C — Dashboard Pro (structure adopted, dark sidebar rejected)

Architecture — Data Flow

Data Flow Architecture



Scan Pipeline

Photo → Dropbox → scan.py → OFF lookup
→ cat mapper → inventory.json

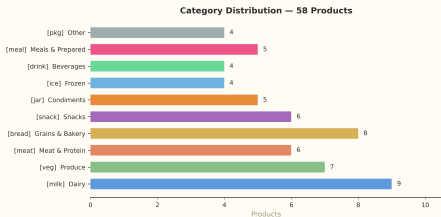
Dashboard Stack

Single-file HTML + vanilla JS
Python server at :9002
60s auto-refresh poll

API Endpoints

GET /api/pantry
PATCH /api/pantry/<bc>
POST /api/pantry
DELETE /api/pantry/<bc>

Category System — 10 Pantry Categories



1. Take categories string from OFF
(e.g. *"Dairies, Milks, Cow milks"*)
2. Split by comma, check each segment
against keyword table
3. First match wins (most specific)
4. No match or empty → "Other"

category_source: "auto" — re-scan can update
category_source: "manual" — user override
preserved across all future scans

38/58 products had OFF taxonomy data → auto-mapped.
Remainder assigned "Other" and manually corrected.

Category UI — Filter Pills, Badges, Dropdown



Mobile: horizontal pill scroll with emoji + name + count



Category dropdown picker (replaces blind cycling)

Location — Pantry / Fridge / Freezer / Spice Rack

Category — pill bar (10 categories + All)

Search — text substring match

All three stack: e.g. *Fridge* + *Dairy* + “*organic*” shows only organic dairy items in the fridge.

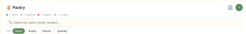
Each card shows a colored pill:



Tap badge → dropdown to override category

Desktop Layout — Sidebar Overhaul

Before (v1)



Stretched mobile layout at 1200px

Sidebar (>1024px)

Stat cards → location list → category filters → sort controls. Background: #F8F4EC

Tablet (600–1024px)

Hamburger icon → slide-in sidebar overlay. Preserves mobile pill UX.

After (v2)



280px sticky sidebar + 2–3 column card grid

Mobile (<600px)

Single column. Location tabs + category pill scroll. No sidebar. Zero regression from v1.

Polish Features — Every Edge Case Handled

Loading — skeleton shimmer cards (gray animated rectangles in grid)

Empty — centered illustration + “Your pantry is empty” + Add CTA

Error — warm amber tone, Retry button calls `loadPantry()` directly

Out of stock — `qty=0` → **Out of stock** badge + muted card

Location: dropdown select (not blind cycle)

Sort: **A→Z** / **Z→A** direction toggle

Stats as clickable filters (tap “Expiring” → filter view)

Null nutrition fields hidden (not shown as “null”)



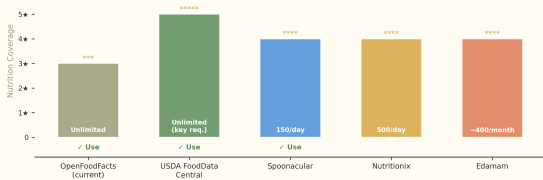
Empty state (left) and error state (right)



Stats as interactive filters in sidebar

Nutrition API Research — Closing Coverage Gaps

Nutrition API Comparison



OpenFoodFacts is community-maintained. Many US grocery products have sparse nutrition data: calories, fiber, and vitamins frequently null.

1. **OpenFoodFacts** (current, primary) — best international coverage
2. **USDA FoodData Central** (fallback 1) — 100% free, unlimited, lab-validated, 250K branded foods
3. **Spoonacular** (fallback 2) — clean UPC endpoint, 150/day free, sufficient for household scan volume

Skip: Edamam (free tier too restrictive), Nutritionix (redundant with USDA).

Design System — Color Tokens & Typography

Design System — Color Tokens

Core Palette		Category Colors	
#F2F2F3	Background	[milk] Dairy	
#88C3A	Sage Green	[veg] Produce	
#F9F0D8	Card BG	[meat] Meat & Protein	
#889580	Warm Gray	[bread] Grains & Bakery	
#2C2C2C	Text	[snack] Snacks	
#88887A	Muted	[jar] Condiments	
		[ice] Frozen	
		[drink] Beverages	
		[meal] Meals & Prepared	
		[pkg] Other	

UI font: System sans-serif stack

Base: 14px body, 16px card names

Card header: 0.95rem semi-bold

Badges/pills: 0.7rem bold, uppercase

Sidebar labels: 0.8rem muted

Cards: border-radius: 12px

Pills/badges: border-radius: 20px

Sidebar: border-radius: 0 (full height)

Card shadow:
0 2px 8px rgba(0,0,0,0.06)

Grid gap: 16px

Technical Decisions

No build step — edit, reload, done

No dependency rot over time

58 products: DOM manipulation trivially fast

Serve as static file or tiny Python server

Any LLM can read/edit the whole thing in context

One file = one deploy artifact

No asset pipeline, no CDN needed

Trivial to version in git

Dashboard is a personal tool, not a product — simplicity wins

1. Write full phase spec before building
2. Builder agent implements the phase
3. Evaluator agent takes Playwright screenshots at 320/420/768/1200/1920px
4. Score each spec requirement: Pass / Partial / Fail
5. Iterate until 100% match
6. Gate to next phase

Key insight: separate builder from evaluator — different optimization targets. Builder optimizes for shipping; evaluator for spec compliance.

Playwright used headless on server (no display required):

```
npx playwright install chromium
```

Results — 5 Phases, 100% Spec Match



Phase 4 final — desktop at 1200px

- 10-category system (auto + manual override)
- 280px sticky sidebar + hamburger on tablet
- Category badges on every card (colored pills)
- Location dropdown (replaces blind cycling)
- Sort direction toggle (A→Z / Z→A)
- Stats as clickable filters
- Skeleton + empty + error states
- Out-of-stock badge at qty=0
- Null nutrition fields hidden

What's Next — v3 Roadmap

Integrate USDA FoodData Central as fallback #1 in `scan.py`. Add Spoonacular as fallback #2.

Target: eliminate null nutrition fields for US branded products.

Effort: 1 sprint. API keys already identified, both free.

Connect current inventory to a recipe API (Edamam or Spoonacular).

Given {items in pantry} → suggest recipes that use them.

Display in new “Recipes” tab on dashboard.

Effort: 2 sprints. Requires recipe API integration.

Track items that hit `qty=0` (out of stock).

Auto-generate shopping list from out-of-stock + low-stock items.

Export as shareable link or push notification.

Effort: 1 sprint. Data already available in `inventory.json`.

Stack remains: single-file HTML + vanilla JS + Python server. No framework migration planned. Every new feature will follow the same Build-Eval-Iterate pattern with explicit spec before implementation.